Bubble Sort and Selection Sort

Programming and Data Structures — Week 5, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement bubble sort and selection sort entirely from scratch; trace both algorithms completely on the same numeric input, directly comparing their step-by-step behavior; prove both run in $O(n^2)$ time by counting comparisons exactly rather than citing the result; and identify the one meaningful practical difference between them — the number of swaps performed — which matters when swapping is an expensive operation.

Outline

Today covers a card-sorting analogy motivating two different sorting strategies; bubble sort, which repeatedly swaps adjacent out-of-order pairs; a complete trace of bubble sort; selection sort, which repeatedly places the correct minimum into position; a complete trace of selection sort; a rigorous comparison-counting argument proving both run in $O(n^2)$; the one meaningful practical difference between them, the number of swaps performed; an in-class quiz; and a summary.

The card-sorting analogy

Picture a hand of playing cards spread face-up on a table in no particular order. One natural strategy compares neighboring cards, swaps them if they are out of order, and repeats this sweep across the whole hand again and again until nothing changes — this is bubble sort. A second natural strategy scans the whole unsorted portion to find the smallest remaining card and moves it directly to the front, then repeats on the remainder — this is selection sort. Both strategies eventually produce a sorted hand; this lecture asks precisely how much work each one does to get there.

Bubble sort, in code

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break        # already sorted - stop early
    return arr
```

The inner loop compares every adjacent pair in the unsorted portion and swaps them if out of order. A key property: after each complete inner-loop pass, the largest remaining element has necessarily "bubbled" all the way to its correct final position at the end of the unsorted portion — which is exactly why the outer loop can shrink the range it checks by one each time (`n - 1 - i`). The `swapped` flag is an important practical optimization: if an entire pass makes no swaps at all, the array is already fully sorted, and the algorithm can stop early rather than needlessly completing every remaining pass.

Tracing bubble sort completely on `[5, 2, 4, 1, 3]`

```
Pass 1: 5,2→swap  2,5,4,1,3 | 5,4→swap  2,4,5,1,3 | 5,1→swap  2,4,1,5,3 | 5,3→swap  2,4,1,3,5
Pass 2: 2,4 ok | 4,1→swap  2,1,4,3,5 | 4,3→swap  2,1,3,4,5 | (4,5 not checked, already placed
Pass 3: 2,1→swap  1,2,3,4,5 | 2,3 ok | (rest already placed)
Pass 4: 1,2 ok | no swaps at all → swapped=False → STOP EARLY
```

Tracing bubble sort completely on `[5, 2, 4, 1, 3]`: Pass 1 makes four swaps and correctly bubbles 5 (the largest) to the very end, leaving `[2, 4, 1, 3, 5]`. Pass 2 checks only the first four elements (since position 4 already holds its correct final value), making two swaps and placing 4 correctly, leaving `[2, 1, 3, 4, 5]`. Pass 3 checks only the first three elements, makes one swap, and produces the fully sorted `[1, 2, 3, 4, 5]`. Pass 4 checks the first two elements, finds them already in order, sets no swaps, and the `swapped` flag correctly triggers an early stop rather than needlessly running the remaining pass.

Selection sort, in code

```python
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Selection sort takes a structurally different approach: for each position `i` in turn, it scans the entire remaining unsorted portion to find the index of the minimum value, then performs exactly one swap to place that minimum directly into position `i`. Unlike bubble sort, there is no possibility of an early stop here — the inner loop must always scan every remaining element to be certain it has found the true minimum, even if the array happens to already be sorted.

Tracing selection sort completely on `[5, 2, 4, 1, 3]`

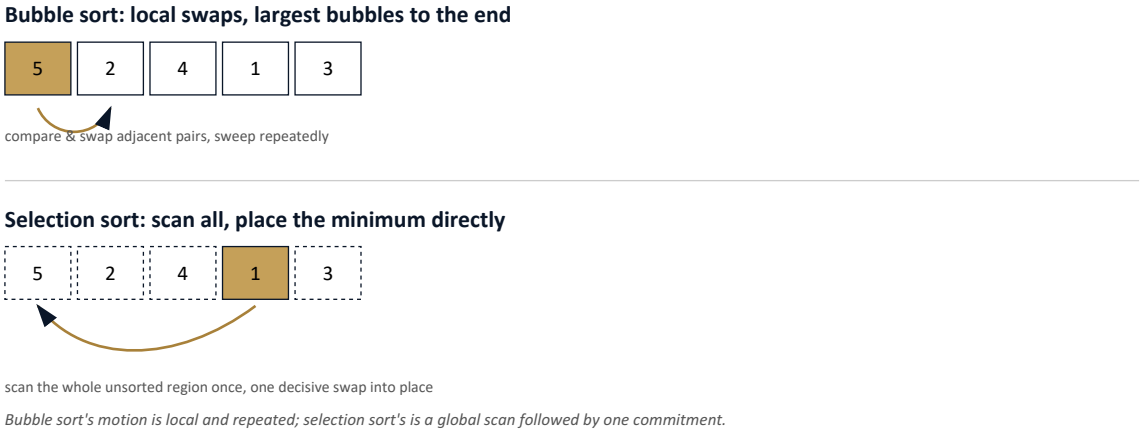
```
i=0: scan [5,2,4,1,3], min is 1 at index 3 → swap arr[0],arr[3] → [1,2,4,5,3]
i=1: scan [2,4,5,3] (from index 1), min is 2 at index 1 → no swap needed → [1,2,4,5,3]
i=2: scan [4,5,3] (from index 2), min is 3 at index 4 → swap arr[2],arr[4] → [1,2,3,5,4]
i=3: scan [5,4] (from index 3), min is 4 at index 4 → swap arr[3],arr[4] → [1,2,3,4,5]
Done (i=4 is the last remaining element, already in place)
```

Tracing selection sort on the identical input `[5, 2, 4, 1, 3]`: at `i=0`, scanning the whole array finds 1 as the minimum, at index 3, and swaps it into position 0. At `i=1`, scanning from index 1 onward finds 2 already at index 1, requiring no swap. At `i=2`, scanning finds 3 at index 4, swapping it into position 2. At `i=3`, scanning finds 4 at index 4, swapping it into position 3, completing the sort. Notice this took exactly 3 actual swaps, compared to bubble sort's 7 swaps on the same input — a difference explored formally later in this lecture.

Why "sorting in place" matters

Both bubble sort and selection sort are in-place algorithms: they rearrange the input array's own elements directly, never allocating a second array to hold intermediate results. The only extra memory used, beyond the input itself, is a small, fixed number of loop variables and index trackers — O(1) additional space, independent of n. This is worth contrasting directly with an approach that builds an entirely new sorted array as it goes, which would need O(n) additional space — a distinction that becomes important again in Week 6, where merge sort's O(n) extra-space requirement is a genuine trade-off against its improved time complexity.

The picture: two different motions



Bubble sort's motion is local and repeated; selection sort's is a global scan followed by one commitment.

Bubble sort's motion is local — only ever comparing and swapping neighbors, with large elements gradually bubbling toward the end over many passes. Selection sort's motion is global within the unsorted region — scanning the entire remaining portion once per outer iteration, but committing to only one decisive swap as a result.

Counting comparisons: proving O(n²), rigorously

Selection sort's total comparison count can be derived exactly, not merely bounded. The inner loop at `i=0` performs $n-1$ comparisons (scanning everything after position 0); at `i=1`, it performs $n-2$ comparisons; and so on, down to 1 comparison at `i=n-2`. Summing this arithmetic series: $(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$, using the same summation formula introduced when analyzing dynamic array resizing costs in Week 2. This is exactly $\Theta(n^2)$, a precise closed-form result, not a rough upper bound — selection sort performs exactly $\frac{n(n-1)}{2}$ comparisons on *every* input, sorted or not, since the inner loop never has an early-exit condition.

Counting comparisons for bubble sort

Bubble sort's worst-case comparison count, when the array is in reverse order (forcing every possible pass with no early stop), is identical to selection sort's: $\frac{n(n-1)}{2}$, since the nested loop structure is the same shape. Its best case, however, differs sharply: on an already-sorted array, the very first pass makes $n-1$ comparisons, finds zero swaps, and the `swapped` flag correctly triggers an immediate stop — giving O(n) total comparisons, genuinely linear. This best-case behavior is structurally impossible for selection sort to replicate, since its inner loop always scans to find the true minimum regardless of whether the array happens to already be sorted.

The one real practical difference: counting swaps

While both algorithms share the same O(n²) comparison count in the worst case, they differ sharply in swap count, which is the one meaningful practical distinction between them. Bubble sort performed 7 swaps on the traced example; selection sort performed only 3. This is not a coincidence of this particular input: selection sort performs at most $n - 1$ swaps in total, always, exactly one per outer-loop iteration, regardless of how disordered the input is. Bubble sort's swap count can be as high as $\frac{n(n-1)}{2}$ in the worst case. When each swap is expensive — moving large records, or writing to slow storage — selection sort's bounded swap count can matter far more than the shared O(n²) comparison count suggests.

MTech addition — stability

A sorting algorithm is called stable if two elements that compare as equal retain their original relative order in the output — a property that matters whenever the sorted values carry additional information beyond the key being compared, such as sorting a list of student records by grade while wanting ties broken by original submission order. Bubble sort is stable: it only ever swaps strictly out-of-order adjacent elements (using >, never >=), so two equal elements are never swapped past each other. Selection sort, as implemented in this lecture, is not guaranteed stable — swapping the found minimum directly into position `i` can move it past an equal-valued element that was originally closer to the front, disturbing their relative order.

MTech addition — a stability example, traced

Tracing three records with a tie on the sort key: `(2,"a")`, `(2,"b")`, `(1,"c")`, sorted by the first value only. Selection sort finds `(1,"c")` as the minimum and swaps it directly with the first position, which happens to swap `(2,"a")` all the way to the last position — the two tied records, `"a"` and `"b"`, end up with their relative order reversed. Bubble sort, restricted to adjacent swaps only, never has a reason to swap `(2,"a")` past `(2,"b")`, since they are never compared as strictly out-of-order relative to each other, preserving their original order. This concrete example demonstrates why the stability property is not just theoretical.

Exercise

As an exercise, trace bubble sort's early-stop behavior on the already-sorted input `[1, 2, 3, 4, 5]`, confirming that exactly one pass occurs, with zero swaps, before the `swapped` flag correctly halts further passes. Separately, modify `selection_sort` to instead find the *maximum* remaining element on each outer iteration and place it directly at the *end* of the unsorted region, rather than finding the minimum and placing it at the front. Trace your modified version on `[5, 2, 4, 1, 3]` and confirm it produces the same sorted result via a mirrored strategy.

Where these algorithms actually appear in practice

Neither algorithm dominates general-purpose sorting in practice — Week 6's advanced sorts are used almost everywhere large or general data must be sorted efficiently. But each has a genuine, specific niche. Bubble sort's early-stop check is a legitimate and efficient O(n) way to verify whether an array is already sorted, independent of using it as a full sorting routine. Selection sort's guarantee of at most $n - 1$ swaps, regardless of input, is genuinely useful when writes are far more expensive than reads or comparisons — flash memory storage, for instance, physically wears out with each write, making an algorithm with a hard cap on write operations valuable even at the cost of more comparisons.

Quiz — count the comparisons

Using the exact comparison-count formula derived in this lecture, work out how many total comparisons selection sort performs on an array of 6 elements, before checking the answer.

A second worked example, side by side

A second worked example, `[3, 3, 1]`, checks behavior around a tie. Bubble sort compares the two 3s with strict >, finds them not out of order, and performs no swap between them — preserving their relative order throughout. Selection sort finds the minimum, 1, and swaps it directly with index 0, which in this particular case happens to preserve the two 3s' relative order as well, since they were both already positioned after the minimum. This illustrates that selection sort's instability is not guaranteed to manifest on every input with ties — only on inputs where the swap mechanics happen to cross a tied pair, as the previous slide's three-record example specifically constructed.

Quiz — answer

Substituting $n = 6$ into $\frac{n(n-1)}{2}$ gives $\frac{6\times5}{2} = 15$ comparisons, exactly. This count is identical for every possible 6-element input — already sorted, reverse-sorted, or in any random order — since selection sort's inner loop has no early-exit condition and always scans the full remaining range to confirm the true minimum. This is a genuinely different behavior from bubble sort, whose comparison count *does* depend on the input, ranging from O(n) on an already-sorted array up to the same $\frac{n(n-1)}{2}$ in the worst case.

Summary

Both bubble sort and selection sort run in $O(n^2)$ time in the worst case, a result proven in this lecture by exact comparison counting rather than simply asserted. Bubble sort can achieve O(n) best-case performance via its early-stop optimization on an already-sorted input, a capability selection sort's

inner loop structurally cannot replicate. Selection sort performs at most $n - 1$ swaps regardless of input, while bubble sort's swap count depends heavily on how disordered the input already is — the one genuinely meaningful practical difference between the two algorithms. Neither algorithm is used in production sorting libraries in practice, but both exist in this course to build the comparison-counting and tracing skills the rest of this unit depends on. Next lecture covers insertion sort, the one simple sort that genuinely is used in practice — for small arrays, or as the final finishing step inside more sophisticated algorithms.